PointerPro Animate

Generated by Doxygen 1.9.8

1 Data Structure Index 3
1.1 Data Structures . . . 3

2 Data Structure Documentation 5
2.1 canvas Struct Reference . . . 5
2.1.1 Detailed Description . . . 5
2.2 Channel Union Reference . . . 5
2.2.1 Detailed Description . . . 6
2.2.2 Field Documentation . . . 6
2.2.2.1 A . . . 6
2.2.2.2 B . . . 6
2.2.2.3 [struct] . . . 6
2.2.2.4 G . . . 6
2.2.2.5 R . . . 6
2.2.2.6 raw . . . 6
2.3 circular_list Struct Reference . . . 7
2.3.1 Detailed Description . . . 7
2.4 list_node Struct Reference . . . 7
2.4.1 Detailed Description . . . 7
2.5 params Struct Reference . . . 7
2.5.1 Detailed Description . . . 7
2.6 sprite Struct Reference . . . 8
2.6.1 Detailed Description . . . 8
2.7 sprite_placement Struct Reference . . . 8
2.7.1 Detailed Description . . . 8

3 File Documentation 9
3.1 animate.h File Reference . . . 9
3.1.1 Detailed Description . . . 10
3.1.2 Typedef Documentation . . . 11
3.1.2.1 animate_fn . . . 11
3.1.2.2 color_t . . . 11
3.1.3 Function Documentation . . . 11
3.1.3.1 animate_color_argb() . . . 11
3.1.3.2 animate_color_rgb() . . . 11
3.1.3.3 animate_create_canvas() . . . 11
3.1.3.4 animate_create_circle() . . . 12
3.1.3.5 animate_create_rectangle() . . . 12
3.1.3.6 animate_create_sprite() . . . 13
3.1.3.7 animate_destroy_canvas() . . . 13
3.1.3.8 animate_destroy_placement() . . . 13
3.1.3.9 animate_destroy_sprite() . . . 14
3.1.3.10 animate_frame_size_bytes() . . . 14

3.1.3.11 animate_generate_frame() . . . 14
3.1.3.12 animate_place_sprite() . . . 15
3.1.3.13 animate_placement_bottom() . . . 15
3.1.3.14 animate_placement_down() . . . 15
3.1.3.15 animate_placement_top() . . . 15
3.1.3.16 animate_placement_up() . . . 16
3.1.3.17 animate_set_animation_function() . . . 16
3.1.3.18 animate_set_animation_params() . . . 16
3.2 animate.h . . . 17
3.3 include/canvas_helper.h File Reference . . . 18
3.3.1 Detailed Description . . . 19
3.3.2 Enumeration Type Documentation . . . 19
3.3.2.1 PlacementMode . . . 19
3.3.3 Function Documentation . . . 19
3.3.3.1 canvas_advance_layer() . . . 19
3.3.3.2 canvas_create_circularlist() . . . 20
3.3.3.3 canvas_destroy_circularlist() . . . 20
3.3.3.4 circularlist_get_listsize() . . . 20
3.3.3.5 circularlist_goto_first() . . . 21
3.3.3.6 circularlist_insert() . . . 21
3.3.3.7 circularlist_move() . . . 21
3.3.3.8 circularlist_remove() . . . 22
3.3.3.9 listnode_get_thislist() . . . 22
3.4 canvas_helper.h . . . 22
3.5 include/misc_helper.h File Reference . . . 23
3.5.1 Detailed Description . . . 24
3.5.2 Macro Definition Documentation . . . 25
3.5.2.1 DBG_PRINT . . . 25
3.5.3 Enumeration Type Documentation . . . 25
3.5.3.1 ErrorType . . . 25
3.5.4 Function Documentation . . . 25
3.5.4.1 validate_dimension() . . . 25
3.5.5 Variable Documentation . . . 26
3.5.5.1 dbg_strings . . . 26
3.6 misc_helper.h . . . 26
3.7 include/physics_helper.h File Reference . . . 26
3.7.1 Detailed Description . . . 27
3.7.2 Function Documentation . . . 27
3.7.2.1 physics_calculate_posx() . . . 27
3.7.2.2 physics_calculate_posy() . . . 28
3.7.2.3 physics_create_params() . . . 28
3.7.2.4 physics_destroy_params() . . . 28

3.7.2.5 physics_get_deltatime() 29
3.7.2.6 physics_set_params() 29
3.8 physics_helper.h 29
3.9 include/pixel_helper.h File Reference 30
3.9.1 Enumeration Type Documentation 31
3.9.1.1 ColorMode 31
3.9.2 Function Documentation 31
3.9.2.1 pixel_get_addr() 31
3.9.2.2 pixel_get_color() 32
3.9.2.3 pixel_get_positive_coord() 32
3.9.2.4 pixel_set_color() 33
3.10 pixel_helper.h 33

Chapter 1

Data Structure Index

1.1 Data Structures

Here are the data structures with brief descriptions:

canvas
Represents the animation canvas 5

Channel
Represents little endian ARGB pixel as a raw 32-bit value word or individual ARGB components in a struct. Assumed Little-Endian ARGB 5

circular_list
Circular list for layering sprite placements 7

list_node
Single node in the layering circular list 7

params
Stores the movement parameters of the sprite, i.e. initial x y velocity, and x y acceleration given by user 7

sprite
Represents a drawable sprite 8

sprite_placement
Represents a placement of reference of sprite 8

Chapter 2

Data Structure Documentation

2.1 canvas Struct Reference

Represents the animation canvas.

```
#include <animate.h>
```

2.1.1 Detailed Description

Represents the animation canvas.

This is where sprite placement will be placed and moved around. Frames will be generated from canvas background and layers of sprite_placements.

The documentation for this struct was generated from the following file:

- animate.h

2.2 Channel Union Reference

Represents little endian ARGB pixel as a raw 32-bit value word or individual ARGB components in a struct. Assumed Little-Endian ARGB.

```
#include <pixel_helper.h>
```

Data Fields

- color_t raw
- struct {
  uint8_t B
  uint8_t G
  uint8_t R
  uint8_t A
  } colors

2.2.1 Detailed Description

Represents little endian ARGB pixel as a raw 32-bit value word or individual ARGB components in a struct. Assumed Little-Endian ARGB.

2.2.2 Field Documentation

2.2.2.1 A

```
uint8_t Channel::A
```

Alpha channel

2.2.2.2 B

```
uint8_t Channel::B
```

Blue channel

2.2.2.3 [struct]

```
struct { ... } Channel::colors
```

Named color components

2.2.2.4 G

```
uint8_t Channel::G
```

Green channel

2.2.2.5 R

```
uint8_t Channel::R
```

Red channel

2.2.2.6 raw

```
color_t Channel::raw
```

Raw color value (0xBBGGRRAA)

The documentation for this union was generated from the following file:

- include/pixel_helper.h

2.3 circular_list Struct Reference

Circular list for layering sprite placements.

```
#include <canvas_helper.h>
```

2.3.1 Detailed Description

Circular list for layering sprite placements.

This implementation mimics the data structure stack. Newly inserted placements go to the end of the list. This means the head of the list is the bottom layer, while tail of the list is the top layer in canvas.

The documentation for this struct was generated from the following file:

- include/canvas_helper.h

2.4 list_node Struct Reference

Single node in the layering circular list.

```
#include <canvas_helper.h>
```

2.4.1 Detailed Description

Single node in the layering circular list.

A node represents a layer of sprite placement.

The documentation for this struct was generated from the following file:

- include/canvas_helper.h

2.5 params Struct Reference

Stores the movement parameters of the sprite, i.e. initial x y velocity, and x y acceleration given by user.

```
#include <physics_helper.h>
```

2.5.1 Detailed Description

Stores the movement parameters of the sprite, i.e. initial x y velocity, and x y acceleration given by user.

The documentation for this struct was generated from the following file:

- include/physics_helper.h

2.6 sprite Struct Reference

Represents a drawable sprite.

```
#include <animate.h>
```

2.6.1 Detailed Description

Represents a drawable sprite.

This is the sprite of the bitmap file loaded, circle, or rectangle. One may reference a sprite many times, i.e. creating multiple sprite placements.

The documentation for this struct was generated from the following file:

- animate.h

2.7 sprite_placement Struct Reference

Represents a placement of reference of sprite.

```
#include <animate.h>
```

2.7.1 Detailed Description

Represents a placement of reference of sprite.

This is the reference of a sprite that can be placed on canvas and moved around. Sprites stack on each other, so they form layers, and they can be moved UP, DOWN, TOP, BOTTOM.

The documentation for this struct was generated from the following file:

- animate.h

Chapter 3

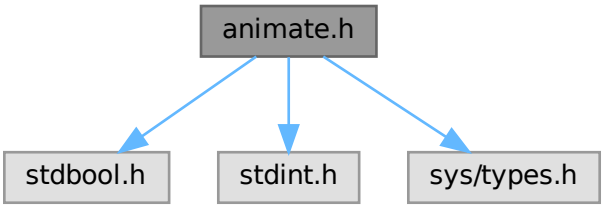
File Documentation

3.1 animate.h File Reference

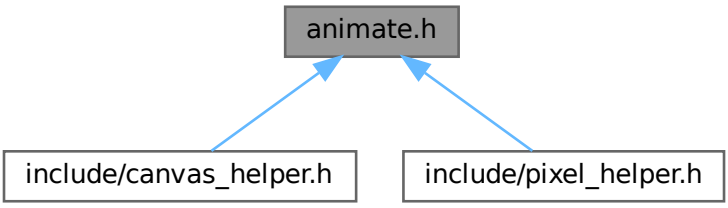
The API of this engine.

```
#include <stdbool.h>
#include <stdint.h>
#include <sys/types.h>
```

Include dependency graph for animate.h:



This graph shows which files directly or indirectly include this file:



Typedefs

- typedef uint32_t color_t

  Representation of a pixel.
- typedef void(∗ animate_fn) (void ∗priv, ssize_t ∗x, ssize_t ∗y, float t)

  Custom animation function for optional extension.

Functions

- static color_t animate_color_rgb (unsigned r, unsigned g, unsigned b)

  Helper function for creating assigning an rgb value to a pixel.
- static color_t animate_color_argb (unsigned a, unsigned r, unsigned g, unsigned b)

  Helper function for creating assigning an argb value to a pixel.
- struct canvas ∗ animate_create_canvas (size_t height, size_t width, color_t background_color)

  Create the canvas on which our animation will be built.
- void animate_destroy_canvas (struct canvas ∗canvas)

  Clean up a canvas.
- struct sprite ∗ animate_create_sprite (const char ∗file)

  Create a sprite from the provided input bitmap file.
- struct sprite ∗ animate_create_rectangle (size_t width, size_t height, color_t c, bool filled)

  Create a sprite that represents a rectangle.
- struct sprite ∗ animate_create_circle (size_t radius, color_t c, bool filled)

  Create a sprite that represents a circle.
- bool animate_destroy_sprite (struct sprite ∗sprite)

  Frees all memory associated with a sprite.
- struct sprite_placement ∗ animate_place_sprite (struct canvas ∗canvas, struct sprite ∗sprite, ssize_t x, ssize_t y)

  Places a sprite on the top layer of the canvas.
- void animate_placement_up (struct sprite_placement ∗sprite_placement)

  Raise a sprite one level towards the top layer.
- void animate_placement_down (struct sprite_placement ∗sprite_placement)

  Lower a sprite one level towards the bottom layer.
- void animate_placement_top (struct sprite_placement ∗sprite_placement)

  Raise the sprite to the top-most layer.
- void animate_placement_bottom (struct sprite_placement ∗sprite_placement)

  Lower the sprite to the bottom-most layer.
- void animate_destroy_placement (struct sprite_placement ∗sprite_placement)

  Removes the placement from its associated canvas and free resources.
- void animate_set_animation_params (struct sprite_placement ∗sprite_placement, ssize_t vx, ssize_t vy, ssize_t ax, ssize_t ay)

  Configure sprite animation.
- void animate_set_animation_function (struct sprite_placement ∗sprite_placement, animate_fn fn, void ∗priv)

  [Optional extension] define animation using the provided function pointer
- size_t animate_frame_size_bytes (struct canvas ∗canvas)

  Return the size in bytes of an animation frame.
- void animate_generate_frame (const struct canvas ∗canvas, size_t frame, size_t frame_rate, void ∗buf)

  Generate a frame of the animation.

3.1.1 Detailed Description

The API of this engine.

3.1.2 Typedef Documentation

3.1.2.1 animate_fn

```
typedef void(* animate_fn) (void *priv, ssize_t *x, ssize_t *y, float t)
```

Custom animation function for optional extension.

Parameters

priv	a handle to the private data provided when the function was registered
x	a reference to the starting position. On return, this should contain the target sprite position for the frame in question.
y	a reference to the starting position. On return, this should contain the position for the frame in question.
t	the current time since the start of the animation.

3.1.2.2 color_t

```
typedef uint32_t color_t
```

Representation of a pixel.

3.1.3 Function Documentation

3.1.3.1 animate_color_argb()

```
static color_t animate_color_argb (
            unsigned a,
            unsigned r,
            unsigned g,
            unsigned b )  [inline], [static]
```

Helper function for creating assigning an argb value to a pixel.

3.1.3.2 animate_color_rgb()

```
static color_t animate_color_rgb (
            unsigned r,
            unsigned g,
            unsigned b )  [inline], [static]
```

Helper function for creating assigning an rgb value to a pixel.

3.1.3.3 animate_create_canvas()

```
struct canvas * animate_create_canvas (
            size_t height,
            size_t width,
            color_t background_color )
```

Create the canvas on which our animation will be built.

Parameters

height	height of the drawing area in pixels
width	width of the drawing area in pixels
background_color	the colour to use when no sprites overlap a pixel.

Returns

Abstract data type for the canvas. The memory associated this struct is allocated by this function and cleaned up by the corresponding destroy function

3.1.3.4 animate_create_circle()

```
struct sprite * animate_create_circle (
            size_t radius,
            color_t c,
            bool filled )
```

Create a sprite that represents a circle.

We currently only support filled circles. The circle pixels can be determined by the following formula: x∗x + y∗y <= r∗r Where x is the x offset from the center of the circle y is the y offset from the center of the circle r is the radius of the circle

Parameters

radius	the radius of the circle
c	the colour of the circle
filled	reserved for future use

Returns

A handle to the created sprite

3.1.3.5 animate_create_rectangle()

```
struct sprite * animate_create_rectangle (
            size_t width,
            size_t height,
            color_t c,
            bool filled )
```

Create a sprite that represents a rectangle.

Parameters

width	the width of the rectangle
height	the height of the rectangle
c	the colour of the rectangle
filled	true if the rectangle should be filled, otherwise false

Returns

A handle to the created sprite

3.1.3.6 animate_create_sprite()

```
struct sprite * animate_create_sprite (
            const char * file )
```

Create a sprite from the provided input bitmap file.

The file format can be assumed to be ARGB32. The bitmap content may by loaded only on first access, however it must only be loaded once. NOTE: the order of image rows differs to what we expect. The first pixel in the bmp file format is on the bottom left while the last pixel is the top right. For simplicity, reverse the order of rows while loading the bitmap.

Parameters

file	the name of the bitmap file to be loaded

Returns

Sprite instance. This then can be used as a reference for placements.

3.1.3.7 animate_destroy_canvas()

```
void animate_destroy_canvas (
            struct canvas * canvas )
```

Clean up a canvas.

Frees all memory associated with the canvas, including placements, but not sprites

Parameters

canvas	the canvas to clean up

3.1.3.8 animate_destroy_placement()

```
void animate_destroy_placement (
            struct sprite_placement * sprite_placement )
```

Removes the placement from its associated canvas and free resources.

Parameters

sprite_placement	a handle to the placement

3.1.3.9 animate_destroy_sprite()

```
bool animate_destroy_sprite (
            struct sprite * sprite )
```

Frees all memory associated with a sprite.

Parameters

sprite	the sprite to clean up

Returns

0 on success, otherwise an error occurred. The only error possible is that the sprite is still in use

3.1.3.10 animate_frame_size_bytes()

```
size_t animate_frame_size_bytes (
            struct canvas * canvas )
```

Return the size in bytes of an animation frame.

This is useful for the caller to determine how much memory to allocate when combining sprites into a frame.

Parameters

canvas	the canvas in question

Returns

the size, in bytes, of a frame for this canvas

3.1.3.11 animate_generate_frame()

```
void animate_generate_frame (
            const struct canvas * canvas,
            size_t frame,
            size_t frame_rate,
            void * buf )
```

Generate a frame of the animation.

Parameters

canvas	the canvas describing the animation
frame	the frame number (starting from 0)
frame_rate	the number of frames per second in the animation
buf	a buffer provided by the user that is large enough to hold a frame. This buffer will contain the frame data on return.

3.1.3.12 animate_place_sprite()

```
struct sprite_placement * animate_place_sprite (
            struct canvas * canvas,
            struct sprite * sprite,
            ssize_t x,
            ssize_t y )
```

Places a sprite on the top layer of the canvas.

Parameters

canvas	the canvas to place the sprite on
sprite	the sprite to place
x	the x coordinate of the canvas to which the top left pixel of the sprite should be placed
y	the y coordinate of the canvas to which the top left pixel of the sprite should be placed

Returns

A handle to a sprite placement.

3.1.3.13 animate_placement_bottom()

```
void animate_placement_bottom (
            struct sprite_placement * sprite_placement )
```

Lower the sprite to the bottom-most layer.

Parameters

sprite_placement	a handle to the placement

3.1.3.14 animate_placement_down()

```
void animate_placement_down (
            struct sprite_placement * sprite_placement )
```

Lower a sprite one level towards the bottom layer.

Parameters

sprite_placement	a handle to the placement

3.1.3.15 animate_placement_top()

```
void animate_placement_top (
            struct sprite_placement * sprite_placement )
```

Raise the sprite to the top-most layer.

Parameters

sprite_placement	a handle to the placement

3.1.3.16 animate_placement_up()

```
void animate_placement_up (
            struct sprite_placement * sprite_placement )
```

Raise a sprite one level towards the top layer.

Parameters

sprite_placement	a handle to the placement

3.1.3.17 animate_set_animation_function()

```
void animate_set_animation_function (
            struct sprite_placement * sprite_placement,
            animate_fn fn,
            void * priv )
```

[Optional extension] define animation using the provided function pointer

Parameters

sprite_placement	a handle to the sprite placement
fn	a custom function to call when determining the position of the sprite in each frame.
priv	private data that is passed to the provided function on call.

3.1.3.18 animate_set_animation_params()

```
void animate_set_animation_params (
            struct sprite_placement * sprite_placement,
            ssize_t vx,
            ssize_t vy,
            ssize_t ax,
            ssize_t ay )
```

Configure sprite animation.

Currently, we support only physics simulation (velocity and acceleration)

Parameters

sprite_placement	a handle to the sprite placement

Parameters

vx	the initial velocity in the x direction
vy	the initial velocity in the y direction
ax	the acceleration in the x direction
ay	the acceleration in the y direction

3.2 animate.h

Go to the documentation of this file.

```
#ifndef ANIMATE_H
#define ANIMATE_H

#include <stdbool.h>   /* for bool           */
#include <stdint.h>    /* for uint32_t       */
#include <sys/types.h> /* for size_t/ssize_t */

/*
 * Abstract data types
 * Exposing the content of these structs allows users to
 * make assumptions on underlying implementation details
 * If we improve the implementation, many users may need
 * to modify their code as the assumptions no longer hold.
 */

struct sprite;

struct canvas;

struct sprite_placement;

typedef uint32_t color_t;

typedef void (*animate_fn)(void* priv, ssize_t* x, ssize_t* y, float t);

// Inline functions are often preferred over macros as types will be checked
static inline color_t animate_color_rgb(unsigned r, unsigned g, unsigned b)
{
    return ((r & 0xff) << 16) | ((g & 0xff) << 8) | ((b & 0xff) << 0);
}

static inline color_t animate_color_argb(unsigned a,
                                         unsigned r, unsigned g, unsigned b)
{
    return animate_color_rgb(r, g, b) | ((a & 0xff) << 24);
}

struct canvas* animate_create_canvas(size_t height, size_t width,
                                     color_t background_color);

void animate_destroy_canvas(struct canvas* canvas);

struct sprite* animate_create_sprite(const char* file);

struct sprite* animate_create_rectangle(size_t width, size_t height, color_t c,
                                        bool filled);

struct sprite* animate_create_circle(size_t radius, color_t c, bool filled);

bool animate_destroy_sprite(struct sprite* sprite);

struct sprite_placement* animate_place_sprite(struct canvas* canvas,
                                              struct sprite* sprite,
                                              ssize_t x, ssize_t y);

void animate_placement_up(struct sprite_placement* sprite_placement);

void animate_placement_down(struct sprite_placement* sprite_placement);

void animate_placement_top(struct sprite_placement* sprite_placement);

void animate_placement_bottom(struct sprite_placement* sprite_placement);

void animate_destroy_placement(struct sprite_placement* sprite_placement);

```

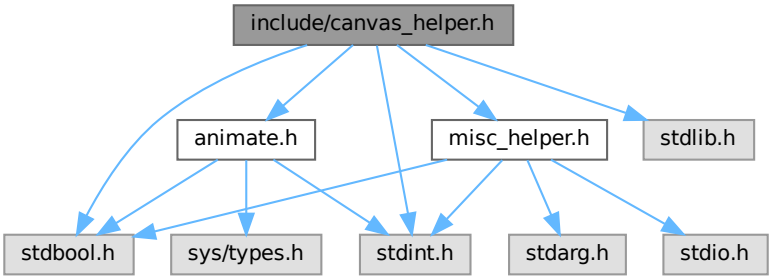
```
00210
00221 void animate_set_animation_params(struct sprite_placement* sprite_placement,
00222                                   ssize_t vx, ssize_t vy,
00223                                   ssize_t ax, ssize_t ay);
00224
00234 void animate_set_animation_function(struct sprite_placement* sprite_placement,
00235                                     animate_fn fn, void* priv);
00236
00245 size_t animate_frame_size_bytes(struct canvas* canvas);
00246
00257 void animate_generate_frame(const struct canvas* canvas,
00258                            size_t frame, size_t frame_rate,
00259                            void* buf);
00260
00261
00262 #endif /* ANIMATE_H */
```

3.3 include/canvas_helper.h File Reference

Everything related to layering sprite placements.

```
#include "animate.h"
#include "misc_helper.h"
#include <stdbool.h>
#include <stdint.h>
#include <stdlib.h>
```

Include dependency graph for canvas_helper.h:



Enumerations

- enum PlacementMode {
  TOP ,
  BOTTOM ,
  MOVEUP ,
  MOVEDOWN }

  Enum for moving sprite_placements.

Functions

- struct circular_list ∗ canvas_create_circularlist ()

  Creates an empty circularlist, without nodes.
- void canvas_destroy_circularlist (struct circular_list ∗circlist)

Destroy everything in a circular list, i.e. all placements.

- struct sprite_placement * canvas_advance_layer (struct circular_list *circlist)

  Moves to the next layer in a circular list.
- void circularlist_goto_first (struct circular_list *circlist)

  Go to the first layer of the layering.
- struct list_node * circularlist_insert (struct circular_list *circlist, struct sprite_placement *data, PlacementMode mode)

  Insert a placement as a new node in a circular list.
- bool circularlist_move (struct circular_list *circlist, struct list_node *listnode, PlacementMode mode)

  Move a layer in a circular list.
- bool circularlist_remove (struct list_node *listnode, struct circular_list *circlist)

  Remove a layer in a circular list, without destroying placement.
- uint32_t circularlist_get_listsize (struct circular_list *circlist)

  Get the number of layers of a circular list.
- struct circular_list * listnode_get_thislist (struct list_node *listnode)

  Get the circular list where a node sits.

3.3.1 Detailed Description

Everything related to layering sprite placements.

3.3.2 Enumeration Type Documentation

3.3.2.1 PlacementMode

```
enum PlacementMode
```

Enum for moving sprite_placements.

Enumerator

TOP	Top layer
BOTTOM	Bottom layer
MOVEUP	Up a layer
MOVEDOWN	Down a layer

3.3.3 Function Documentation

3.3.3.1 canvas_advance_layer()

```
struct sprite_placement * canvas_advance_layer (
            struct circular_list * circlist )
```

Moves to the next layer in a circular list.

Parameters

circlist	The circular list to be advanced

Returns

The sprite placement before advancing

3.3.3.2 canvas_create_circularlist()

```
struct circular_list * canvas_create_circularlist ( )
```

Creates an empty circularlist, without nodes.

Returns

The circular list

3.3.3.3 canvas_destroy_circularlist()

```
void canvas_destroy_circularlist (
            struct circular_list * circlist )
```

Destroy everything in a circular list, i.e. all placements.

Parameters

circlist	The circular list to be destroyed

3.3.3.4 circularlist_get_listsize()

```
uint32_t circularlist_get_listsize (
            struct circular_list * circlist )
```

Get the number of layers of a circular list.

Parameters

circlist	The circular list in question

Returns

Size of the circular list

3.3.3.5 circularlist_goto_first()

```
void circularlist_goto_first (
            struct circular_list * circlist )
```

Go to the first layer of the layering.

Parameters

circlist	The circular list to be modified

3.3.3.6 circularlist_insert()

```
struct list_node * circularlist_insert (
            struct circular_list * circlist,
            struct sprite_placement * data,
            PlacementMode mode )
```

Insert a placement as a new node in a circular list.

Parameters

circlist	The circular list to be inserted
data	Reference of sprite placement to be put
mode	PlacementMode enum, should use TOP

Returns

The new node

3.3.3.7 circularlist_move()

```
bool circularlist_move (
            struct circular_list * circlist,
            struct list_node * listnode,
            PlacementMode mode )
```

Move a layer in a circular list.

Parameters

circlist	The circular list to be modified
listnode	The node to be moved
mode	PlacementMode enum

Returns

true when successful

false when failed

3.3.3.8 circularlist_remove()

```
bool circularlist_remove (
            struct list_node * listnode,
            struct circular_list * circlist )
```

Remove a layer in a circular list, without destroying placement.

Parameters

listnode	The node to be removed
circlist	The circular list to be modified

Returns

true when successful

false when failed

3.3.3.9 listnode_get_thislist()

```
struct circular_list * listnode_get_thislist (
            struct list_node * listnode )
```

Get the circular list where a node sits.

Parameters

listnode	The node in question

Returns

The circular list of the node

3.4 canvas_helper.h

Go to the documentation of this file.

```
00001
00005 #ifndef CANVAS_HELPER_H
00006 #define CANVAS_HELPER_H
00007
00008 #include "animate.h"
00009 #include "misc_helper.h"
00010 #include <stdbool.h>
00011 #include <stdint.h>
00012 #include <stdlib.h>
00013
00017 typedef enum {
00018     TOP,
00019     BOTTOM,
00020     MOVEUP,
00021     MOVEDOWN
00022 } PlacementMode;
00023
00024
00034 struct circular_list;
00035
```

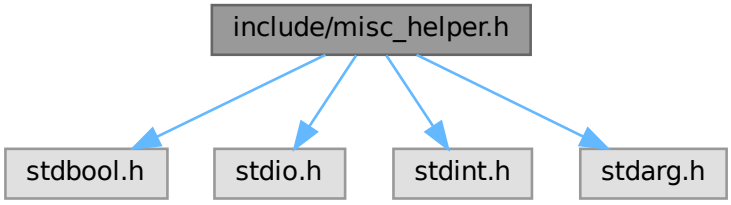
```
struct list_node;


struct circular_list* canvas_create_circularlist();

void canvas_destroy_circularlist(struct circular_list* circlist);

struct sprite_placement* canvas_advance_layer(struct circular_list* circlist);


void circularlist_goto_first(struct circular_list* circlist);

struct list_node* circularlist_insert(struct circular_list* circlist,
                                      struct sprite_placement* data,
                                      PlacementMode mode);

bool circularlist_move(struct circular_list* circlist,
                       struct list_node* listnode,
                       PlacementMode mode);

bool circularlist_remove(struct list_node* listnode,
                         struct circular_list* circlist);


uint32_t circularlist_get_listsize(struct circular_list* circlist);

struct circular_list* listnode_get_thislist(struct list_node* listnode);

#endif /* CANVAS_HELPER_H */
```

3.5 include/misc_helper.h File Reference

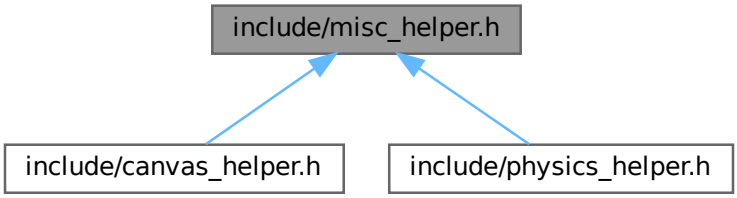
Helpers.

```
#include <stdbool.h>
#include <stdio.h>
#include <stdint.h>
#include <stdarg.h>
```

Include dependency graph for misc_helper.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define DBG_PRINT(code, ...) dbg_print_helper(code, __VA_ARGS__)

  Prints debug info.

Enumerations

- enum ErrorType {
  ERR_MALLOC ,
  ERR_MALLOC_WSIZE ,
  ERR_IO ,
  ERR_BM_HEADER ,
  ERR_BM_V5 ,
  FREED ,
  INVALID_ARG ,
  INVALID_DIM ,
  TOOBIG ,
  FAIL ,
  SUCCESS ,
  CUSTOM }

  Enum for debug printing.

Functions

- bool validate_dimension (size_t height, size_t width)

  Validates the size of area. Prevent crazy allocations.

Variables

- const char *const dbg_strings [ ]

  String to be printed in debug printing.

3.5.1 Detailed Description

Helpers.

3.5.2 Macro Definition Documentation

3.5.2.1 DBG_PRINT

```
#define DBG_PRINT(
            code,
            ...  ) dbg_print_helper(code, __VA_ARGS__)
```

Prints debug info.

3.5.3 Enumeration Type Documentation

3.5.3.1 ErrorType

```
enum ErrorType
```

Enum for debug printing.

Enumerator

ERR_MALLOC	Malloc error
ERR_MALLOC_WSIZE	Malloc error, with size
ERR_IO	I/O error
ERR_BM_HEADER	Bitmap header error
ERR_BM_V5	Bitmap V5 header error
FREED	Memory allocation freed
INVALID_ARG	Invalid argument
INVALID_DIM	Invalid dimension
TOOBIG	Size is too big
FAIL	Task is unsuccessful
SUCCESS	Task is successful
CUSTOM	Custom debug message

3.5.4 Function Documentation

3.5.4.1 validate_dimension()

```
bool validate_dimension (
            size_t height,
            size_t width )
```

Validates the size of area. Prevent crazy allocations.

Parameters

height	Height in pixels
width	Width in pixels

Returns

true if valid

false if invalid

3.5.5 Variable Documentation

3.5.5.1 dbg_strings

```
const char* const dbg_strings[]  [extern]
```

String to be printed in debug printing.

3.6 misc_helper.h

Go to the documentation of this file.

```
#ifndef MISC_HELPER_H
#define MISC_HELPER_H

#include <stdbool.h>
#include <stdio.h>
#include <stdint.h>

typedef enum {
    ERR_MALLOC,
    ERR_MALLOC_WSIZE,
    ERR_IO,
    ERR_BM_HEADER,
    ERR_BM_V5,
    FREED,
    INVALID_ARG,
    INVALID_DIM,
    TOOBIG,
    FAIL,
    SUCCESS,
    CUSTOM
} ErrorType;

extern const char* const dbg_strings[];

// https://edstem.org/au/courses/31567/discussion/3160179
#ifdef DEBUG
#include <stdarg.h>
void dbg_print_helper(ErrorType code, ...);
#define DBG_PRINT(code, ...) dbg_print_helper(code, __VA_ARGS__)
#else
#define DBG_PRINT(code, ...) do {} while(0)
#endif

bool validate_dimension(size_t height, size_t width);

#endif /* MISC_HELPER_H */
```

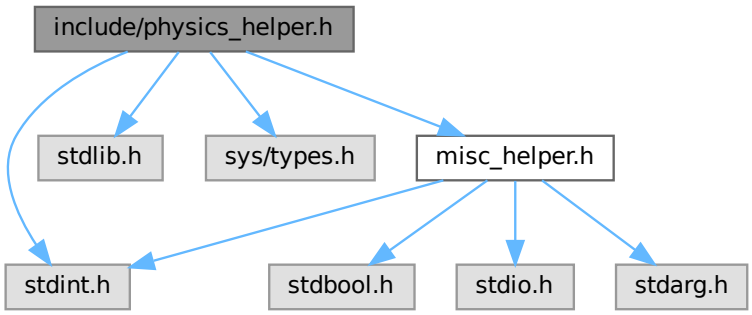
3.7 include/physics_helper.h File Reference

Physics calculations.

```
#include <stdint.h>
#include <stdlib.h>
#include <sys/types.h>
```

```
#include "misc_helper.h"
```

Include dependency graph for physics_helper.h:



Functions

- struct params ∗ physics_create_params ()

  Creates an empty parameter storage.
- void physics_destroy_params (struct params ∗params)

  Destroy a parameter storage.
- void physics_set_params (struct params ∗params, ssize_t initial_x, ssize_t initial_y, ssize_t x_velocity, ssize↩_t y_velocity, ssize_t x_acceleration, ssize_t y_acceleration)

  Sets the parameters given by user.
- ssize_t physics_calculate_posx (ssize_t initial_x, struct params ∗params, double delta_time)

  Calculates the current x position of a sprite placement.
- ssize_t physics_calculate_posy (ssize_t initial_y, struct params ∗params, double delta_time)

  Calculates the current y position of a sprite placement.
- double physics_get_deltatime (size_t frame, size_t frame_rate)

  Calculates time passed since t0 (first frame)

3.7.1 Detailed Description

Physics calculations.

Pixel and pixel grid helper.

3.7.2 Function Documentation

3.7.2.1 physics_calculate_posx()

```
ssize_t physics_calculate_posx (
            ssize_t initial_x,
            struct params * params,
            double delta_time )
```

Calculates the current x position of a sprite placement.

Parameters

initial_x	initial x position
params	The parameter storage
delta_time	time passed since t0 (first frame)

Returns

Current x position

3.7.2.2 physics_calculate_posy()

```
ssize_t physics_calculate_posy (
            ssize_t initial_y,
            struct params * params,
            double delta_time )
```

Calculates the current y position of a sprite placement.

Parameters

initial_x	initial y position
params	The parameter storage
delta_time	time passed since t0 (first frame)

Returns

Current y position

3.7.2.3 physics_create_params()

```
struct params * physics_create_params ( )
```

Creates an empty parameter storage.

Returns

The new parameter storage

3.7.2.4 physics_destroy_params()

```
void physics_destroy_params (
            struct params * params )
```

Destroy a parameter storage.

Parameters

params	The parameter storage to be destroyed

3.7.2.5 physics_get_deltatime()

```
double physics_get_deltatime (
            size_t frame,
            size_t frame_rate )
```

Calculates time passed since t0 (first frame)

Parameters

frame	Frame number
frame_rate	Frame rate of the intended animation

Returns

Time passed since t0 (first frame)

3.7.2.6 physics_set_params()

```
void physics_set_params (
            struct params * params,
            ssize_t initial_x,
            ssize_t initial_y,
            ssize_t x_velocity,
            ssize_t y_velocity,
            ssize_t x_acceleration,
            ssize_t y_acceleration )
```

Sets the parameters given by user.

Parameters

params	The parameter storage
initial_x	initial x position
initial_y	initial y position
x_velocity	initial x velocity
y_velocity	initial y velocity
x_acceleration	x acceleration
y_acceleration	y acceleration

3.8 physics_helper.h

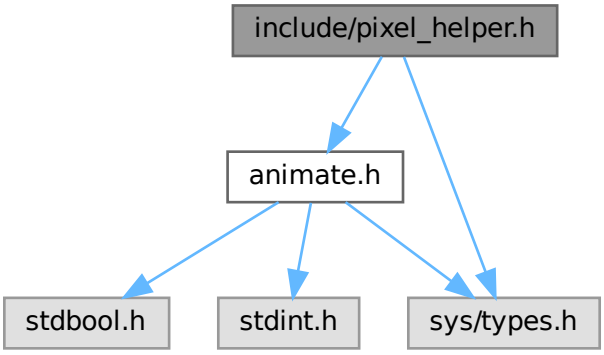
Go to the documentation of this file.

```
#ifndef PHYSICS_HELPER_H
#define PHYSICS_HELPER_H

#include <stdint.h>
#include <stdlib.h>
#include <sys/types.h>
#include "misc_helper.h"

struct params;


struct params* physics_create_params();

void physics_destroy_params(struct params* params);


void physics_set_params(struct params* params,
                        ssize_t initial_x, ssize_t initial_y,
                        ssize_t x_velocity, ssize_t y_velocity,
                        ssize_t x_acceleration, ssize_t y_acceleration);


ssize_t physics_calculate_posx(ssize_t initial_x,
                               struct params* params,
                               double delta_time);

ssize_t physics_calculate_posy(ssize_t initial_y,
                               struct params* params,
                               double delta_time);

double physics_get_deltatime(size_t frame, size_t frame_rate);

#endif /* PHYSICS_HELPER_H */
```

3.9 include/pixel_helper.h File Reference

```
#include "animate.h"
#include <sys/types.h>
```

Include dependency graph for pixel_helper.h:



Data Structures

- union Channel

  Represents little endian ARGB pixel as a raw 32-bit value word or individual ARGB components in a struct. Assumed Little-Endian ARGB.

Enumerations

- enum ColorMode {
  NORMAL ,
  FORCE_ALPHA }

  Enum for pixel coloring.

Functions

- static color_t * pixel_get_addr (color_t *grid, ssize_t x, ssize_t y, size_t height, size_t width)

  Get the address of a pixel (x, y) in a pixel grid.
- static color_t pixel_get_color (color_t *grid, ssize_t x, ssize_t y, size_t height, size_t width)

  Get the color of a pixel (x, y) in a pixel grid.
- static void pixel_set_color (color_t *grid, color_t value, ssize_t x, ssize_t y, size_t height, size_t width, ColorMode mode)

  Set color of a pixel (x, y) in a pixel grid.
- static ssize_t pixel_get_positive_coord (ssize_t coord, size_t length)

  Get coordinate for edge warping purpose.

3.9.1 Enumeration Type Documentation

3.9.1.1 ColorMode

```
enum ColorMode
```

Enum for pixel coloring.

Enumerator

NORMAL	Leave alpha channel value as is
FORCE_ALPHA	Force non-zero alpha to be 0xFF

3.9.2 Function Documentation

3.9.2.1 pixel_get_addr()

```
static color_t * pixel_get_addr (
            color_t * grid,
            ssize_t x,
            ssize_t y,
            size_t height,
            size_t width )  [inline], [static]
```

Get the address of a pixel (x, y) in a pixel grid.

Parameters

grid	The pixel grid in question

Parameters

x	x coordinate
y	y coordinate
height	Height of the grid
width	Width of the grid

Returns

The address of the pixel

3.9.2.2 pixel_get_color()

```
static color_t pixel_get_color (
            color_t * grid,
            ssize_t x,
            ssize_t y,
            size_t height,
            size_t width )  [inline], [static]
```

Get the color of a pixel (x, y) in a pixel grid.

Parameters

grid	The pixel grid in question
x	x coordinate
y	y coordinate
height	Height of the grid
width	Width of the grid

Returns

The color of the pixel

3.9.2.3 pixel_get_positive_coord()

```
static ssize_t pixel_get_positive_coord (
            ssize_t coord,
            size_t length )  [inline], [static]
```

Get coordinate for edge warping purpose.

Parameters

coord	coordinate
length	Max width along the coordinate

Returns

Coordinate in range [0, Max width)

3.9.2.4 pixel_set_color()

```
static void pixel_set_color (
            color_t * grid,
            color_t value,
            ssize_t x,
            ssize_t y,
            size_t height,
            size_t width,
            ColorMode mode )  [inline], [static]
```

Set color of a pixel (x, y) in a pixel grid.

Parameters

grid	The pixel grid in question
value	The color value to be set
x	x coordinate
y	y coordinate
height	Height of the grid
width	Width of the grid
mode	ColorMode enum

3.10 pixel_helper.h

Go to the documentation of this file.

```
#ifndef PIXEL_HELPER_H
#define PIXEL_HELPER_H

#include "animate.h"
#include <sys/types.h>

typedef union {
    color_t raw;
    struct {
        uint8_t B;
        uint8_t G;
        uint8_t R;
        uint8_t A;
    } colors;
} Channel;

typedef enum {
    NORMAL,
    FORCE_ALPHA
} ColorMode;


static inline color_t* pixel_get_addr(color_t* grid,
                                      ssize_t x, ssize_t y,
                                      size_t height, size_t width)
{
    return &grid[x + (y * width)];
}

static inline color_t pixel_get_color(color_t* grid,
                                      ssize_t x, ssize_t y,
```

```
                                        size_t height, size_t width)
{
    return *pixel_get_addr(grid, x, y, height, width);
}

static inline void pixel_set_color(color_t* grid,
                                   color_t value,
                                   ssize_t x, ssize_t y,
                                   size_t height, size_t width,
                                   ColorMode mode)
{
    if (mode == FORCE_ALPHA) {
        value |= 0xFF « 24;
    }
    *pixel_get_addr(grid, x, y, height, width) = value;
}

static inline ssize_t pixel_get_positive_coord(ssize_t coord, size_t length)
{
    if (!length) { return 0; }
    ssize_t res = coord % (ssize_t)length;
    return (res < 0) ? (res + (ssize_t)length) : res;
}

#endif /* PIXEL_HELPER_H */
```